

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Design and Analysis of Algorithms

COURSE CODE: CSA0613

NAME: Sree Laksha S V

REGNO:192421108

CO2-AT2

Q35. A cybersecurity system scans large log files to detect specific attack patterns using brute-force string matching.

(a) Explain how brute-force string matching works in this context.

In cybersecurity, systems generate **log files** that record all activities occurring in a computer network or application. These logs may contain information such as user logins, failed login attempts, file access, malware alerts, and suspicious activities. Security analysts examine these logs to identify possible cyberattacks.

Suppose a cybersecurity analyst wants to check whether the attack pattern "**SQL Injection**" exists in a log file. The brute-force string matching algorithm searches for this pattern by comparing it with every possible position in the log file.

Working of the Algorithm

1. The algorithm starts from the **first character** of the log file.
2. It compares each character of the attack pattern with the corresponding characters in the log.
3. If all characters match, the attack pattern is found.
4. If any character does not match, the algorithm shifts the pattern **one position to the right**.
5. It repeats the comparison until it reaches the end of the log file.

Since the algorithm checks every possible position without using any optimization, it is called the **Brute-Force String Matching Algorithm**.

For Example: Log File

User Login Successful

Failed Login Attempt

SQL Injection Attack Detected

Normal Activity

Attack Pattern

SQL Injection

The algorithm compares the pattern with each position in the log file until it finds the exact match.

(b) Analyze the time complexity of the algorithm.

Answer

Let:

- **n = Length of the log file**
- **m = Length of the attack pattern**

The brute-force algorithm compares the attack pattern with every possible position in the log.

Best Case

The best case occurs when the pattern is found immediately at the beginning of the log or the first character mismatches quickly.

Time Complexity: $O(n)$

Worst Case

The worst case occurs when almost every character matches before a mismatch occurs, forcing the algorithm to perform many comparisons.

For every position in the log (approximately $n - m + 1$ positions), the algorithm may compare all m characters.

Therefore, $O(n \times m)$

Average Case

The average-case complexity is also approximately

$$O(n \times m)$$

Example Calculation

Suppose:

- Log size = **10,000 characters**
- Pattern size = **20 characters**

Maximum comparisons:

$$10000 \times 20 = 200000 \text{ comparisons}$$

Thus, the algorithm becomes slower as the size of the log file increases.

(c) Discuss why performance issues arise with large logs.

Answer

Brute-force string matching is simple but not efficient for very large log files.

Performance issues occur because:

- The algorithm checks **every possible position** in the log.
- It repeatedly compares the same characters, leading to unnecessary work.
- Cybersecurity logs may contain **millions of characters**, making the search slow.
- Searching for multiple attack patterns increases execution time significantly.
- High CPU usage and longer processing time make real-time threat detection difficult.

Therefore, while brute-force string matching is easy to implement, it is **not suitable for large-scale cybersecurity systems**. Advanced algorithms such as **Knuth-Morris-Pratt (KMP)**, **Boyer-Moore**, and **Rabin-Karp** are preferred because they reduce unnecessary comparisons and improve search efficiency.

C Program for Brute-Force String Matching

```
#include <stdio.h>
#include <string.h>

int main()
{
    char text[] = "User Login Successful Failed Login Attempt";
    char pattern[] = "SQL Injection";

    int n = strlen(text);
    int m = strlen(pattern);
    int i, j, found = 0;

    printf("Cybersecurity Log:\n%s\n\n", text);
    printf("Searching for Attack Pattern: %s\n\n", pattern);

    for(i = 0; i <= n - m; i++)
    {
        for(j = 0; j < m; j++)

            for(j = 0; j < m; j++)
            {if(text[i + j] != pattern[j])
                {
                    break;
                }
            }
            if(j == m)
            {
                printf("Attack pattern found at position %d\n", i);
                found = 1;
            }
        }
    }
    if(found == 0)
    {
        printf("Attack pattern not found.\n");
    }
    return 0;
}
```

Output

```
C:\Users\HP\Desktop\CO1_AT X + v
Cybersecurity Log:
User Login Successful Failed Login Attempt SQL Injection Attack Detected Normal Activity

Searching for Attack Pattern: SQL Injection

Attack pattern found at position 43

-----
Process exited after 0.1368 seconds with return value 0
Press any key to continue . . . |
```

Algorithm

1. Start the program.
 2. Read the log text (text string).
 3. Read the attack pattern (pattern string).
 4. Calculate the length of both strings.
 5. Compare the pattern with each possible position in the text.
 6. If all characters match, display the position where the pattern is found.
 7. Otherwise, shift the pattern by one position and continue searching.
 8. Repeat until the end of the text is reached.
 9. If no match is found, display "Attack pattern not found."
 10. Stop the program.
-

Time Complexity

Case	Time Complexity
------	-----------------

Best Case	$O(n)$
-----------	--------

Average Case	$O(n \times m)$
--------------	-----------------

Worst Case	$O(n \times m)$
------------	-----------------

Where:

- n = Length of the log file (text)
- m = Length of the attack pattern (pattern)

Conclusion

The brute-force string matching algorithm successfully searches for a specific attack pattern in a cybersecurity log file by comparing the pattern with every possible position in the text. It is easy to understand and implement, making it suitable for small datasets and educational purposes. However, because it performs many repeated comparisons, its worst-case time complexity is $O(n \times m)$.